

Chapitre 2 : Organisation Mémoire

Exercices d'Application (à faire pendant le cours)

Dr. El Hadji Bassirou TOURÉ

Département de Mathématiques et Informatique

Université Cheikh Anta Diop de Dakar

Objectif

Ces exercices sont à réaliser **pendant le cours**, immédiatement après chaque notion. Ils permettent de vérifier la compréhension avant de passer aux TDs.

1 Exercice A1 : Endianness (Little-Endian)

Énoncé

La valeur 0xDEADBEEF est stockée à l'adresse 0x100 sur une machine **little-endian**.
Remplir le tableau montrant le contenu de la mémoire :

Adresse	0x100	0x101	0x102	0x103
Contenu (hex)	_____	_____	_____	_____

Méthode

1. **Little-endian** = l'octet de poids faible (LSB) est stocké à l'adresse la plus basse
2. Pour 0xDEADBEEF : les octets sont DE, AD, BE, EF
3. L'octet de poids faible est EF, il va à l'adresse 0x100
4. Puis BE à 0x101, AD à 0x102, DE à 0x103

À vous !

Sur la même machine little-endian, la valeur 0x12345678 est stockée à l'adresse 0x200.

Adresse	0x200	0x201	0x202	0x203
Contenu	_____	_____	_____	_____

Si on lit un **short** (2 octets) à l'adresse 0x200, quelle valeur obtient-on ? _____

2 Exercice A2 : Pointeurs — Valeur et Adresse

Énoncé

Soit le code suivant sur une machine 32 bits :

```
int x = 42;          /* &x = 0x1000 */
int *p = &x;         /* &p = 0x2000 */
```

Compléter le tableau :

Question	Réponse
Valeur de x	_____
Adresse de x (&x)	_____
Valeur de p	_____
Adresse de p (&p)	_____
Valeur pointée *p	_____
Taille de p (sizeof)	_____

Méthode

Un pointeur a **deux aspects** :

- Sa **valeur** = l'adresse qu'il contient (vers où il pointe)
- Son **adresse** = où il est stocké en mémoire (&p)

L'opérateur * (déréférencement) permet d'accéder à la valeur pointée.

À vous !

Avec le même code, dessiner la mémoire aux adresses 0x1000 et 0x2000 (4 octets chacune, little-endian).

À 0x1000 (variable x = 42 = 0x2A) :

0x1000	0x1001	0x1002	0x1003
_____	_____	_____	_____

À 0x2000 (pointeur p = 0x1000) :

0x2000	0x2001	0x2002	0x2003
_____	_____	_____	_____

3 Exercice A3 : Arithmétique des Pointeurs

Énoncé

Soit le tableau suivant (`int` = 4 octets) :

```
int arr[5] = {10, 20, 30, 40, 50}; /* &arr[0] = 0x1000 */
int *p = arr;
```

Calculer les adresses et valeurs :

Expression	Adresse/Valeur	Calcul
<code>&arr[0]</code>	0x1000	(donné)
<code>&arr[1]</code>	_____	$0x1000 + \text{_____} \times 4$
<code>&arr[3]</code>	_____	_____
<code>p + 2</code>	_____	_____
<code>*(p + 2)</code>	_____	(valeur à cette adresse)
<code>p[4]</code>	_____	équivalent à _____

Formule clé

$$p + n = p + n \times \text{sizeof}(*p)$$

Pour un `int*`, ajouter 1 au pointeur ajoute 4 octets à l'adresse !

Équivalences :

- `arr[i] ≡ *(arr + i)`
- `&arr[i] ≡ arr + i`

À vous !

Soit `short tab[4]` avec `&tab[0] = 0x2000`. (`short` = 2 octets)

<code>&tab[2] =</code>	_____
<code>(char*)tab + 5 =</code>	_____
<code>(int*)tab + 1 =</code>	_____

4 Exercice A4 : Chaînes — strlen vs sizeof

Énoncé

Comparer les deux déclarations suivantes :

```
char arr[] = "UCAD";  
char *ptr = "UCAD";
```

Expression	Pour arr	Pour ptr (64 bits)
sizeof(...)	_____	_____
strlen(...)	_____	_____
Modification [0]='X' possible ?	_____	_____
Réaffectation = "autre" possible ?	_____	_____

Différences clés

- `char arr[]` : **tableau** alloué sur la pile, contenu **modifiable**
- `char *ptr` : **pointeur** vers une chaîne littérale en mémoire **read-only**
- `sizeof(arr)` = taille du tableau (inclut `'\0'`)
- `sizeof(ptr)` = taille du pointeur (4 ou 8 octets selon l'architecture)
- `strlen` = nombre de caractères **avant** le `'\0'`

À vous !

Représenter la chaîne "UCAD" en mémoire (5 octets, avec le terminateur) :

Index	0	1	2	3	4
Caractère	_____	_____	_____	_____	_____
Hex	_____	_____	_____	_____	_____

Table ASCII : U=0x55, C=0x43, A=0x41, D=0x44, NUL=0x00

Récapitulatif des formules

- **Arithmétique pointeurs** : `p + n = p + n × sizeof(*p)`
- **Équivalences** : `p[i] ≡ *(p + i)` et `&p[i] ≡ p + i`
- **Chaînes** : `strlen` compte sans le `'\0'`, `sizeof` compte avec
- **Little-endian** : LSB à l'adresse basse